1.3 Listen und Rekursion

Taxis melden sich in beliebiger Reihenfolge an und ab. Unsere **Liste** kennt nur ihren ersten Knoten (`anfang`), den Rest erledigen die Knoten. *Website: Station 1.3, Projekt Taxizentrale*

Aufgabe 1 *Wie viele stehen hinter dir?*

Die erste Person in der Mensa-Schlange will wissen, wie viele anstehen. Jede darf nur ihren Hintermann fragen.

a) Ergänze die Regel:

„Steht hinter mir _____, antworte ich ____. Sonst frage ich _____ und antworte mit _____."

b) Spielt es zu fünft durch. Wer antwortet zuerst, wer zuletzt?

_____

_____

Aufgabe 2 *Projekt Taxizentrale, TODO 2*

a) Ergänze `laenge()` nach der Regel aus Aufgabe 1. Beschrifte: *Abbruchbedingung*, *rekursiver Aufruf*.

iterativ (Warteschlange, 1.1)

```
public int anzahl() {
   int zaehler = 0;
   Knoten aktuell = anfang;
   while (aktuell != null) {
      zaehler++;
      aktuell = aktuell.getNachfolger();
   }
   return zaehler;
}
```

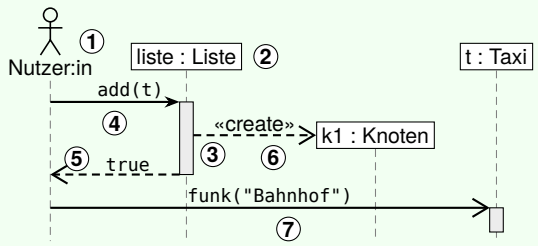
rekursiv (Knoten)

```
public int laenge() {
   if (_____) {    // _____
      return ____;
   } else {
      return _____;  // _____
   }
}
```

rekursiv (Liste)

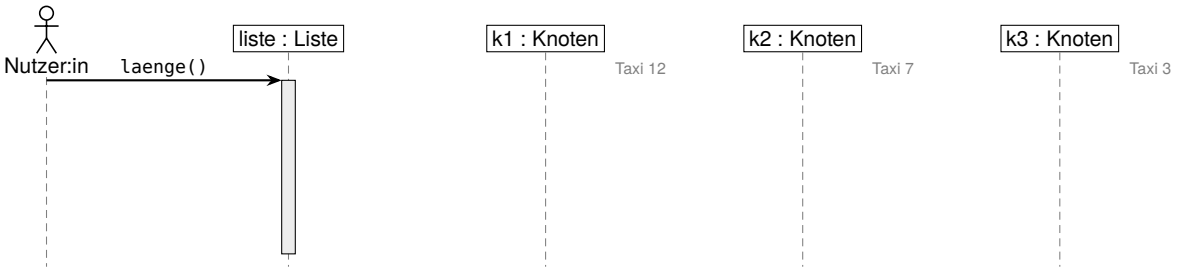
```
public int laenge() {
   if (anfang == null) { return ____; }
   else { return _____; }
}
```

Das UML-Sequenzdiagramm: Wer ruft wann wen auf?



① **Akteur**: startet den Ablauf (im Projekt: Hauptprogramm).
② **Objekt** `name : Klasse` mit **Lebenslinie** (Zeit läuft nach unten).
③ **Aktivierungsbalken**: Objekt arbeitet oder wartet auf Antwort.
④ **Synchroner Aufruf**, gefüllte Spitze: Der Aufrufer wartet auf die Antwort (so jeder Methodenaufruf in Java).
⑤ **Rückgabe**, gestrichelt, mit Wert: Danach endet der Balken.
⑥ **Erzeugung** «create» (`new`): Pfeil zum Kopf des neuen Objekts.
⑦ **Asynchroner Aufruf**, offene Spitze: Der Aufrufer schickt ab und arbeitet sofort weiter, ohne Rückgabe (wie Textnachricht statt Anruf).

b) Ergänze für `liste.laenge()` alle Aufrufe, Rückgaben mit Wert und Aktivierungsbalken.



c) Die Aufrufe sind _____, weil _____.

Rekursion

Eine Methode heißt _____, wenn sie sich selbst aufruft. Ohne _____ ruft sie sich endlos auf. Die Liste ist eine **rekursive Datenstruktur**: Ein Knoten mit seinem Rest ist wieder eine Liste. Darum erledigt jeder Knoten seinen Teil und gibt den _____ an seinen _____ weiter.

Aufgabe 3 *Projekt Taxizentrale, TODO 3*

```
// Knoten: hängt d ans Ende an
public boolean add(DatenElement d) {
   if (__________________) {
      nachfolger = __________________;
      return true;
   } else {
      return ____________________;
   }
}
// Liste
public boolean add(DatenElement d) {
   if (anfang == null) {     // Sonderfall
      anfang = __________________;
      return true;
   } else {
      return __________________;
   }
}
```

a) Ergänze `add` und übertrage es ins Projekt.

b) Der letzte Knoten liefert `true`. Was geschieht mit diesem Wert auf dem Rückweg?

_____

_____

c) Warum braucht die Liste einen Sonderfall, der Knoten aber nicht?

_____

_____

Aufgabe 4 *Projekt Taxizentrale, TODO 4a bis 4c*

a) Gleich oder dasselbe? Sage die Ausgabe vorher:

```
Taxi a = new Taxi(7, "Aylin");
Taxi b = new Taxi(7, "Aylin");
println(a == b);
```

_____

== prüft, ob zwei Verweise auf _____ zeigen. Für **inhaltliche** Gleichheit erweitern wir den Vertrag `DatenElement` (4a):

`boolean` _____`;`

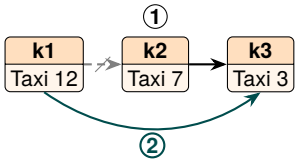
b) Danach meldet der Compiler einen Fehler in `Taxi`. Warum? Ergänze dort (4b):

```
// Taxi (4b)
public boolean istGleich(DatenElement d) {
   return ____________________________
          ____________________________;
}
// Knoten (4c): Ist d ab hier enthalten?
public boolean enthaelt(DatenElement d) {
   if (__________________) {
      return true;
   } else if (__________________) {
      return _______;
   } else {
      return ____________________;
   }
}
```

_____

Aufgabe 5 *Projekt Taxizentrale, TODO 5*

Taxi 7 meldet sich ab. Ein Knoten kann sich nicht selbst aushängen: Den Verweis auf ihn hat sein _____. Deshalb schaut jeder Knoten auf seinen **Nachfolger** voraus.



① k1 prüft die Daten seines Nachfolgers.
② Passen sie, überspringt k1 ihn.

```
// Knoten
public boolean entfernen(DatenElement d) {
   if (nachfolger == null) {
      return false;
   } else if (____________________________) { // ①
      nachfolger = ____________________; // ②
      return true;
   } else {
      return ____________________;
   }
}
```

Den ersten Knoten kann niemand überspringen. Sonderfall in `Liste.entfernen`:

```
if (anfang.getDaten().istGleich(d))
anfang = ____________________;
```

Es geht auch anders – besprecht mündlich, warum man es so machen kann:

- `laenge()` ginge auch mit einer Wiederholung wie `anzahl()`. Was ist leichter zu lesen, was braucht weniger Speicher?
- Die Liste könnte wie die Warteschlange einen Verweis auf das `ende` haben. Was würde `add` dann sparen?
- `entfernen` im Knoten könnte statt `boolean` den Knoten zurückgeben, der danach an seiner Stelle steht (sich selbst oder seinen Nachfolger). Was müsste der Aufrufer mit der Rückgabe tun?